

[Sign Up](#)

DATABASE INDEXING

"Concept & Coding" YT Video Notes

Database Indexing:
(RDBMS)

Monday, 25 September 2023 1:23 PM

I

Few things we have to understand first, before we see how indexing works.

1. How Table data (rows) are actually stored?

2. What type of indexing present?

3. Understanding the data structure used for indexing and how it works?

↑

Then we will combine all and see how actually the DBMS do the indexing....

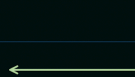
Let's Start...

1. How Table data is actually stored?

	Column1	Column2	Column3
	Emp ID	Name	Address
Row1	1	A	City A
Row2	2	B	City B
Row3	3	C	City C
Row4	4	D	City D

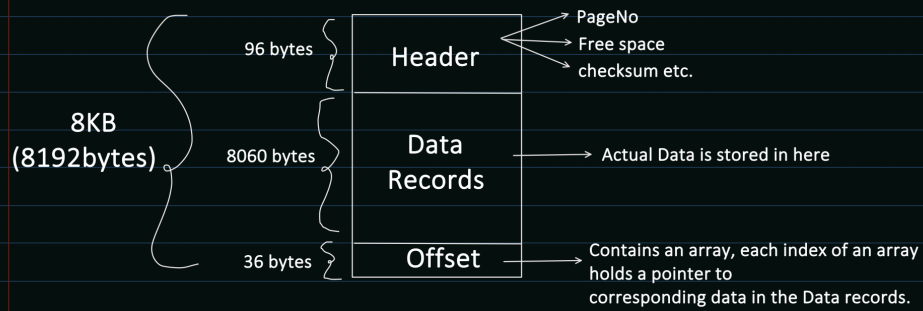
This is just a logical representation.

Actual Data is not ~~X~~ stored in this way.

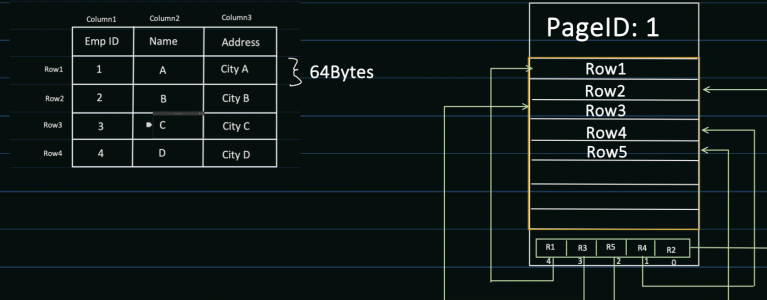


- DBMS creates **Data Pages** (generally its 8KB but depends upon DB to DB)
- Each Data Page can store multiple table rows in it.

Just an example of Data page.



So if 1 table row is lets say 64Bytes, so in 1 data page we can have ~125 DB rows



DBMS creates and manage these data pages. As for storing 1 table data, it can create many data pages.

These data pages ultimately gets stored in the Data Block in physical memory like disk.

What is Data block?

- Data Block is the minimum amount of data which can be read/write by an I/O operation.
- It is managed by underlying storage system like disk. Data Block size can range from 4kb to 32kb (common size is 8KB).
- So based on the data block size, it can hold 1 or many Data page.

DBMS maintains the mapping of DataPage and Data Block.

Data Page1	Data Block 1
Data Page2	Data Block 1
Data Page3	Data Block 2
Data Page4	Data Block 2

Remember, DBMS controls Data pages (like what Row goes in which page or sequence of pages etc.) but has no control on Data Blocks (data blocks can be scattered over the disk)

2. What type of indexing present in RDBMS?



Text Mode

Now before we understand Clustered Indexing and Non-Clustered Indexing, we have to first understand,

What is Indexing?

What Data Structure it uses and how it works?

Indexing:

It is used to increase the performance of the database query. So that data can be fetched faster.

Without indexing, DBMS has to iterate each and every table row to find the requested data.

i.e $O(N)$, if there are millions of rows, query can take some time to fetch the data.

Which Data Structure provides better time complexity than $O(N)$?

B+ Tree, it provides $O(\log N)$ time complexity for insertion, searching & deletion.

How B (Balanced) Tree works?

- It maintains sorted data.
- All leaf are at the same level
- M order B tree means, each node can have at most M childrens.
- And M-1 Keys per node.

Lets store below data in 3-Order B tree:

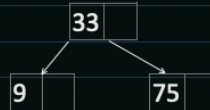
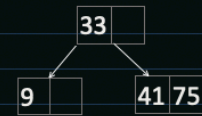
9, 33, 75, 41, 98, 214, 126, 55, 72

Pointer	Key1	Pointer	key2	Pointer
$< \text{key1}$		$\geq \text{key1 and } < \text{key2}$		$\geq \text{key 2}$

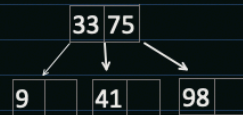
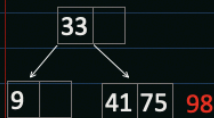
9

9 33

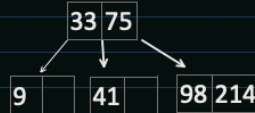
9 33 75



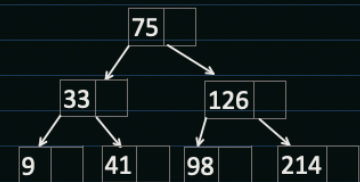
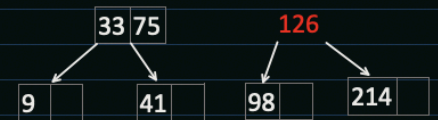
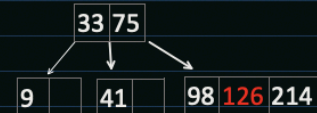
Insert 98



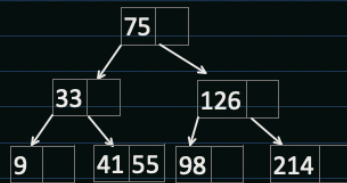
Insert 214



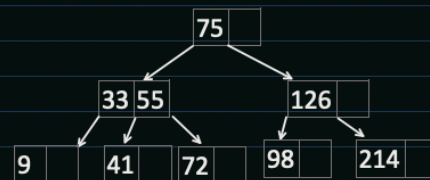
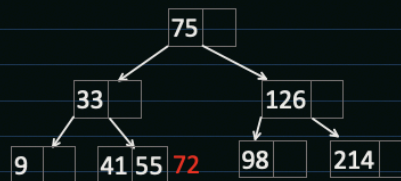
Insert 126



Insert 55



Insert 72

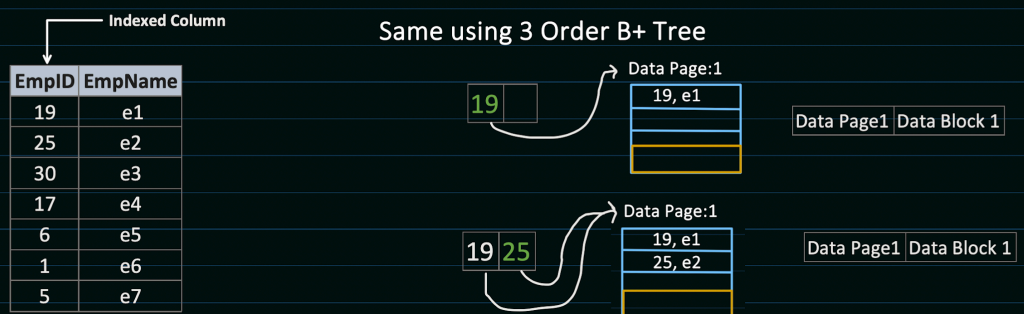


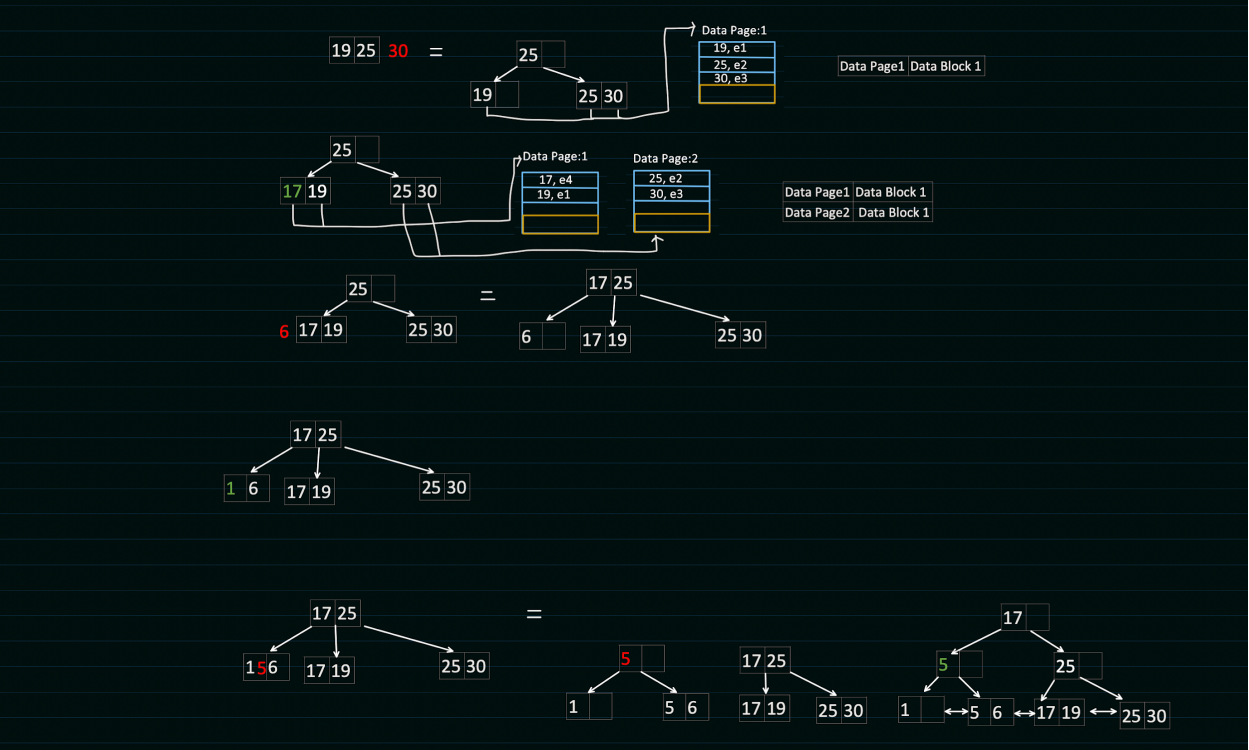
B+ tree is exactly same as B tree, with additional feature like child nodes are also linked to each other.

DBMS uses B+ Tree to manage its Data Pages and Rows within the pages.

Root node or Intermediary node hold the Value which is used for faster searching the data. Possible that value might have deleted from DB, but its can be used for sorting the tree.

Leaf node actually holds the indexed column value.





Indexing

Clustered Indexing

Non-Clustered Indexing

- Order of Rows inside the data pages, match with the order of the Index.

For example:

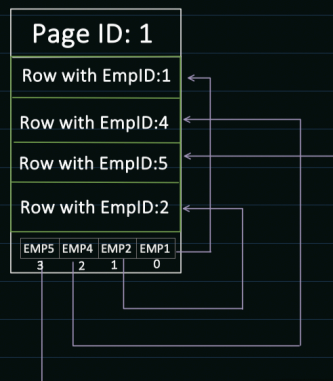
*Lets make it
Primary Key*

EMPID	NAME	ADDRESS
1	A	City1
4	C	City3
5	D	City4
2	B	City2

*when EmpID is marked as Primary key,
DBMS will create an INDEX on it and will
sort the EmpID data to faciliate the
faster search.*

EMPID INDEX
1
2
4
5

Data Page:



So,

- There can be only 1 clustered index present / table bcoz ordering of pages can be done based on one index only.
- If manually you have not specified any Clustered index, then DBMS looks for PRIMARY KEY which is UNIQUE and NOT NULL and use it as a Clustered key.
- If in table there is no PRIMARY KEY available, then internally it creates a hidden Column which is used as Clustered index. (This columns just increase sequentially, so guaranteed unique and not null)